

Универзитет у Београду
Факултет организационих наука
Мастер академске студије
Студијски програм: Електронско пословање
Модул: Технологије електронског пословања

Пројектна документација

**АР паметне учионице — приказ
контекстуалних информација из паметног
окружења у проширеној стварности**

Студент:
Иван Јелисавчић
Број индекса: 2025/3263

Јун 2026.

Садржај

1	Увод у проширену стварност	2
2	ARCore платформа	3
2.1	Праћење покрета	4
2.2	Разумевање окружења	4
2.3	Процена осветљења	4
2.4	Дубина, сидра и објекти праћења	5
2.5	Праћење слика (Augmented Images)	5
3	Архитектура система	6
3.1	Серверска апликација	6
3.2	Контролна табла	7
3.3	АР апликација	7
4	Развој ARCore апликације	8
4.1	Потребна опрема и софтвер на рачунару	8
4.2	Потребна опрема на телефону	8
4.3	Креирање пројекта и зависности	8
4.4	Подешавање манифеста	9
4.5	Кораки до прве АР сцене	10
5	Имплементација АР прегледа учионица	11
5.1	Конфигурација сесије и базе слика	11
5.2	Препознавање учионице	12
5.3	Сидрење панела	12
5.4	Исцртавање панела	13
5.5	Преузимање података у реалном времену	13
5.6	Животни циклус и ослобађање ресурса	14
6	Приказ апликације	15
6.1	Почетни екран	15
6.2	Екран са упутством	15
6.3	АР екран — препознавање маркера	16
6.4	АР екран — усидрени панел	17
6.5	Демонстрација са контролном таблом	17

1 Увод у проширену стварност

Проширена стварност (енгл. *Augmented Reality*, скраћено AR) представља технологију која дигитални садржај — тродимензионалне моделе, текст, слике или анимације — пројектује преко слике стварног света у реалном времену. За разлику од класичних рачунарских приказа, који постоје искључиво на екрану и одвојено од физичког окружења, проширена стварност дигиталне елементе уклапа у простор око корисника тако да они делују као његов саставни део: виртуелна фотеља стоји на стварном поду, дигитална стрелица показује дуж стварне улице, а информациона табла лебди изнад стварних врата учионице. Кључна одлика ове технологије јесте интерактивност у реалном времену — како се корисник креће кроз простор, дигитални садржај се континуирано прилагођава новој перспективи, чиме се ствара утисак да виртуелни објекти заиста постоје у физичком свету.

Проширену стварност треба јасно разликовати од виртуелне стварности (енгл. *Virtual Reality*). Виртуелна стварност корисника у потпуности измешта у синтетичко окружење: стављањем наменских наочара стварни свет нестаје, а замењује га рачунарски генерисана сцена. Проширена стварност, насупрот томе, стварни свет задржава као основу и само га допуњује. Овај однос сликовито описује континуум стварности и виртуелности — замишљена оса на чијем једном крају стоји потпуно стварно окружење, а на другом потпуно виртуелно. Проширена стварност налази се ближе стварном крају континуума, јер виртуелни елементи чине мањи део доживљаја, док виртуелна стварност заузима супротни пол. Између њих се простиру различити облици мешане стварности, у којима се удео дигиталног и физичког садржаја постепено мења.

Иако се проширена стварност доживљава као савремена технологија, њени корени сежу деценијама уназад. Још крајем шездесетих година двадесетог века конструисани су први уређаји који су се носили на глави и пред очи корисника постављали једноставну рачунарску графику уклопљену у поглед на просторију. Ти рани системи били су гломазни, везани каблом за рачунар и доступни искључиво у лабораторијама. Током деведесетих година технологија је нашла примену у војној авијацији и индустријским истраживањима, а сам термин „проширена стварност” ушао је у ширу употребу. Права прекретница, међутим, наступила је појавом паметних телефона: уређај који милијарде људи свакодневно носе у цепу поседује управо оно што је проширеној стварности потребно — камеру, екран, сензоре покрета и довољно снажан процесор. Технологија која је некада захтевала специјализовану и скупу опрему постала је доступна сваком власнику просечног мобилног телефона.

Широка доступност отворила је врата изузетно разноврсним применама. У области игара и забаве глобални феномен постала је игра *Pokémon GO*, која је виртуелна бића сместила у паркове, улице и тргове стварних градова и тиме милионима корисника по први пут приближила идеју проширене стварности. У трговини и опремању дома апликације попут *IKEA Place* омогућавају да се комад намештаја у природној величини „постави” у дневну собу пре куповине, па купац одмах види да ли се нова софа уклапа у простор и бојом и димензијама. У навигацији се упутства за кретање исцртавају директно преко слике улице коју камера снима, тако да пешак уместо апстрактне мапе види стрелицу која лебди тачно изнад раскрснице на којој треба да скрене.

Образовање је још једно подручје у којем проширена стварност показује пуну вредност: уместо статичних илустрација у уџбенику, ученици могу да обиђу тродимензионални модел људског срца који куца на школској клупи или да посматрају Сунчев систем како

кружи изнад катедре. У медицини се технологија користи за визуелизацију вена приликом давања инјекција, као и за навођење хирурга током захвата, при чему се снимци унутрашњих органа пројектују директно преко тела пацијента. У индустрији и одржавању, техничар који поправља сложену машину може кроз екран или паметне наочаре да види упутства за склапање исцртана тачно преко делова на које се односе, чиме се смањује број грешака и скраћује обука.

Поред ових специјализованих домена, проширена стварност незаметно побољшава и свакодневни живот. Камера телефона преводи натписе и јеловнике на страном језику и приказује превод на самој слици, а апликације за пробу наочара или шминке показују како ће производ изгледати на лицу корисника. Посебно занимљиву примену, међутим, отварају паметна окружења: савремене зграде, кампуси и учионице опремљени су сензорима који непрекидно мере температуру, буку и квалитет ваздуха, али ти подаци по правилу остају скривени у удаљеним контролним таблама и извештајима. Проширена стварност нуди далеко природнији приступ тим информацијама — довољно је уперити телефон у врата учионице да би се изнад њих, у стварном простору, приказали живи подаци о тој просторији: да ли је заузета и до када, који се час у њој тренутно одржава и какви су услови за рад. Управо та примена — приказ контекстуалних информација из паметног окружења у проширеној стварности — предмет је апликације описане у овом раду.

Разлог због којег је проширена стварност на паметним телефонима доживела тако брзу експанзију јесте укидање улазних баријера. Кориснику није потребан ниједан додатни уређај: довољан је телефон који већ поседује, а програмерима су на располагању зреле софтверске платформе које сложене проблеме рачунарског вида — праћење положаја уређаја, препознавање површина и процену осветљења — решавају уместо њих. На *Android* платформи централно место међу тим решењима заузима *ARCore*, технологија компаније *Google* на којој је заснована и апликација из овог рада, а која је детаљно представљена у наредном поглављу.

2 ARCore платформа

ARCore је платформа компаније *Google* за развој апликација проширене стварности на *Android* уређајима. Реч је о бесплатном комплету за развој софтвера (*SDK*) који програмерима ставља на располагање готова решења за најтеже проблеме проширене стварности: одређивање положаја уређаја у простору, препознавање површина у окружењу и процену осветљења сцене. Платформа се на уређаје испоручује кроз системску компоненту *Google Play Services for AR*, која се аутоматски ажурира путем продавнице *Google Play*, па апликације увек користе актуелну верзију без потребе да је саме укључују у инсталациони пакет. *ARCore* је подржан на стотинама сертификованих модела телефона и таблета различитих произвођача, при чему сертификација гарантује да су камера, сензори покрета и процесор уређаја довољно квалитетни за стабилно искуство проширене стварности. Захваљујући томе, једна иста апликација ради на огромном броју уређаја који се већ налазе у рукама корисника, без икаквог додатног хардвера.

Суштина платформе може се сажети у три темељне способности: праћење покрета, разумевање окружења и процена осветљења. Свака од њих решава по један предуслов уверљиве проширене стварности, а заједно омогућавају да виртуелни садржај делује као да заиста постоји у физичком простору.

2.1 Праћење покрета

Да би виртуелни објект остало на свом месту док се корисник креће, телефон у сваком тренутку мора да зна где се налази и куда је усмерен. *ARCore* тај проблем решава поступком који се назива визуелно-инерцијална одометрија. Платформа на слици са камере проналази визуелно упечатљиве тачке — ивице, углове и контрастне детаље текстура — и прати како се њихов положај мења из кадра у кадар. Истовремено читава податке са акцелерометра и жироскопа, сензора који мере убрзање и ротацију уређаја. Комбиновањем ова два извора информација, који се међусобно допуњују и исправљају, *ARCore* израчунава положај и оријентацију телефона у простору са шест степени слободе: три трансляције дуж просторних оса и три ротације око њих. Захваљујући томе, виртуелни садржај се у сваком кадру исцртава из тачно одговарајуће перспективе — када корисник приђе виртуелном објекту, објект се увећава; када га заобиђе, види га са друге стране, баш као да је реч о стварном предмету.

2.2 Разумевање окружења

Поред сопственог положаја, уређај мора да разуме и простор око себе. *ARCore* непрекидно анализира распоред карактеристичних тачака које прати и, када уочи групе тачака које леже у истој равни, закључује да је пронашао површину: под, радни сто, седиште столице или зид. Платформа препознаје и хоризонталне и вертикалне равни, одређује њихове границе и проширује их како камера обухвата нове делове простора. Тако изграђен модел окружења омогућава да се виртуелни објекти постављају на смислена места — фигура на под, а слика на зид — уместо да лебде у ваздуху.

Важан механизам који овај модел чини употребљивим из перспективе корисника јесте тестирање погодака (енгл. *hit-testing*). Када корисник додирне екран, *ARCore* из положаја камере кроз додирнуту тачку на екрану пушта замишљени зрак у тродимензионални простор и израчунава где тај зрак сече препознате равни или друге елементе окружења. Резултат је прецизна тачка у простору, са положајем и оријентацијом, на коју апликација може да постави виртуелни садржај. Овај механизам је темељ великог броја АР апликација, јер сваки додир корисника претвара у тачку стварног простора на коју се може везати дигитални садржај.

2.3 Процена осветљења

Трећа способност платформе односи се на светлост. Виртуелни објект исцртан без обзира на услове осветљења у просторији одмах одаје своју вештачку природу: превише је светао у мрачној соби или безличан на јаком сунцу. *ARCore* зато из слике са камере процењује карактеристике осветљења сцене — просечан интензитет, боју светлости, а у режиму *Environmental HDR* и правац главног извора света те околишко осветљење погодено за реалистичне одсјаје. Апликација те податке користи приликом сенчења виртуелних објеката, па они добијају сенке и одсјаје усклађене са стварним окружењем, што значајно доприноси утиску да објект заиста припада сцени.

2.4 Дубина, сидра и објекти праћења

Поред три основне способности, за практичан рад важно је још неколико појмова. *Depth API* омогућава да *ARCore*, користећи само једну обичну камеру, за сваки пиксел слике процени удаљеност од уређаја и тако изгради дубинску мапу сцене. Захваљујући њој, виртуелни садржај се може постављати на произвољне површине, а не само на препознате равни, док виртуелни објекти могу бити заклоњени стварним предметима који се налазе испред њих (оклузија), што додатно појачава уверљивост приказа.

Виртуелни садржај се за стварни свет везује помоћу сидара (енгл. *anchor*). Сидро представља фиксну позу у простору коју платформа активно одржава: како *ARCore* временом прикупља нове информације и прецизира своје разумевање сцене, он коригује и положаје сидара, па садржај причвршћен за њих остаје „прикован” за исто место у стварном свету уместо да постепено отплута. Сидра се по правилу везују за објекте праћења (енгл. *trackable*) — равни, карактеристичне тачке и друге елементе које платформа препознаје и прати кроз време.

У пракси се *ARCore* користи или непосредно, кроз његов изворни програмски интерфејс, или кроз библиотеке вишег нивоа које поједностављују исцртавање тродимензионалног садржаја. Међу њима се издваја *SceneView*, библиотека која обједињује *ARCore* и графички погон *Filament*, па програмер уместо ниског графичког кода ради са сценом, чворовима и моделима, што знатно убрзава развој.

2.5 Праћење слика (Augmented Images)

Поред равни и карактеристичних тачака, *ARCore* уме да препознаје и прати унапред задате дводимензионалне слике у стварном свету — способност позната под називом *Augmented Images* (праћење слика). Програмер унапред саставља базу слика, објекат типа *AugmentedImageDatabase*, у коју сваку референтну слику додаје позивом *addImage* уз јединствено име и, опционо, физичку ширину одштампаног примерка изражену у метрима. Када је база придружена конфигурацији сесије, *ARCore* у сваком кадру упоређује карактеристичне тачке слике са камере са тачкама слика из базе. Чим препозна неку од њих, платформа враћа објекат *AugmentedImage* који носи име препознате слике, њену позу у простору (*centerPose*), процењене димензије (*extentX* и *extentZ*) и стање праћења. Препозната слика тиме постаје пуноправан објекат праћења: за њу се може везати сидро, а виртуелни садржај постављен у њеном координатном систему прати је док се корисник креће.

Квалитет препознавања пресудно зависи од саме слике маркера. Добра референтна слика богата је детаљима, има висок контраст и избегава понављајуће шаре и велике једнобојне површине, јер се препознавање заснива на распореду карактеристичних тачака. *Google* уз платформу испоручује алат *arcoreimg*, који слику оцењује на скали од 0 до 100; препоручује се оцена од најмање 75 да би праћење било поуздано. Навођење физичке ширине слике додатно је важно: када зна стварне димензије маркера, платформа одмах по препознавању тачно процењује његову удаљеност и позу, уместо да их постепено прецизира посматрањем из више углова. Једна база може да садржи и до хиљаду слика, при чему се истовремено прати до двадесет препознатих маркера.

За апликацију из овог рада праћење слика представља природан избор и средишњи механизам читавог решења. Одштампани маркер на вратима учионице једнозначно идентификује просторију — име препознате слике у себи носи ознаку учионице, па апликација

одмах зна од ког сервера затражити податке — док сидро везано за позу слике обезбеђује да информациони панел остане „прикован” изнад врата и када се корисник помера, приближава или мења угао гледања. Поврх свега, платформа ради на широком спектру постојећих уређаја без додатне опреме, чиме апликација постаје доступна најширем кругу корисника. Начин на који се ове могућности конкретно користе у развоју приказан је у наредним поглављима.

3 Архитектура система

Решење описано у овом раду чине три компоненте које заједно симулирају и приказују једно паметно окружење: серверска апликација која игра улогу паметне зграде, веб контролна табла којом се током демонстрације мењају вредности сензора и *Android* АР апликација као главни потрошач података. Ток података је једносмеран и једноставан: контролна табла шаље нове вредности сензора серверу, сервер их чува и обогаћује изведеним информацијама, а АР апликација их периодично преузима и исцртава у простору.

1. Презентер на контролној табли помери клизач или промени заузетост учионице; промена се после кратког задршка шаље серверу.
2. Сервер проверава да ли су вредности у дозвољеним границама, уписује их у меморијско стање и на сваки упит израчунава статусе и препоруку.
3. АР апликација, док прати маркер учионице, на сваке три секунде преузима стање те учионице и освежава панел усидрен изнад маркера.

3.1 Серверска апликација

Сервер је *Spring Boot* апликација (верзија 3.5, Java 21) која симулира паметно окружење. Стање учионица држи у меморији, у структури `ConcurrentHashMap`, без базе података: при покретању се учитавају три учионице (101, 102 и 103) са почетним вредностима температуре, буке и угљен-диоксида, заузетости и дневним распоредом часова. Целокупан изведени садржај рачуна се на серверу, па су оба клијента само прикази добијених података. РЕСТ интерфејс чине три крајње тачке приказане у табели 1.

Метода	Путања	Опис
GET	<code>/api/rooms</code>	листа свих учионица са тренутним стањем
GET	<code>/api/rooms/{roomId}</code>	стање једне учионице; 404 за непознат идентификатор
PUT	<code>/api/rooms/{roomId}/sensors</code>	упис нових вредности сензора и заузетости

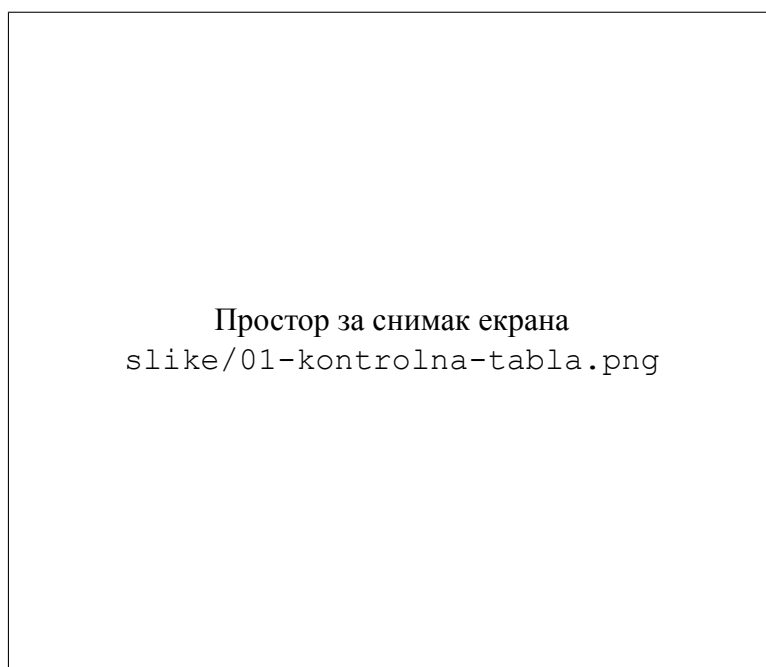
Табела 1: Крајње тачке РЕСТ интерфејса сервера

За сваку измерену величину сервер израчунава статус са вредностима OK, WARNING или CRITICAL према задатим праговима: температура је у реду између 20 и 26 °C, бука испод 45 dB, а угљен-диоксид испод 800 ppm, док вредности изван ширих граница постају критичне. На основу статуса формира се и препорука на српском језику као подршка

одлучивању — од „Услови су оптимални.” преко „Пратити услове у просторији.” до „Хитно проветрити просторију.” када је концентрација угљен-диоксида критична. Сервер из распореда часова и сопственог сата изводи и текст заузетости (назив текућег часа и време до којег је учионица заузета), а враћеним вредностима додаје благи синусни шум како би панели деловали живо и без сталних промена на контролној табли.

3.2 Контролна табла

Контролна табла је једнострана *Angular* апликација (назив `kontrolna-tabla`) са по једном картицом за сваку учионицу. Картица садржи прекидач заузетости и три клизача — температуру, буку и угљен-диоксид — чија се свака промена, после задршка од 300 ms (енгл. *debounce*), шаље серверу позивом `PUT /api/rooms/{roomId}/sensors`. Табла истовремено на сваких пет секунди преузима свежа стање свих учионица. Њена улога у систему је демонстрациона: пошто у учионицама не постоје стварни сензори, презентер преко клизача уживо симулира промене у паметном окружењу и посматра како се оне одражавају на АР панелу.



Слика 1: Контролна табла са картицама учионица и клизачима

3.3 АР апликација

Трећа и централна компонента је *Android* апликација заснована на *ARCore* платформи, главни потрошач података из паметног окружења. Она препознаје маркер на вратима учионице, из његовог имена изводи идентификатор просторије, преузима њено стање са сервера и изнад маркера сидри панел са живим подацима. Развоју и имплементацији ове апликације посвећена су наредна два поглавља.

4 Развој ARCore апликације

Пре имплементације самог AP прегледа учионица потребно је припремити развојно окружење, обезбедити уређај који подржава ARCore и поставити структуру пројекта са неопходним зависностима. У наставку су описани ти кораци, при чему наведене верзије алата и библиотека одговарају стварном пројекту из овог рада.

4.1 Потребна опрема и софтвер на рачунару

Развој се одвија у окружењу *Android Studio*, званичном интегрисаном развојном окружењу за Android платформу. Довољно је инсталирати последњу стабилну верзију, јер она већ садржи одговарајући JDK (верзије 17 или новије) и менаџер за *Android SDK*, кроз који се преузима платформа која одговара вредности `compileSdk = 36` из овог пројекта. Програмски језик је *Kotlin 2.1.20*, у комбинацији са *Android Gradle Plugin*-ом 8.13.0, док се сам *Gradle* не инсталира посебно — пројекат садржи *Gradle wrapper*, скрипту која при првом покретању преузима тачно ону верзију алата са којом је пројекат тестиран.

Када је реч о емулатору, развој AP апликације реално захтева физички уређај: емулатор подржава ARCore само у ограниченим сценаријима, са виртуелном сценом уместо стварног окружења, па се понашање апликације — посебно препознавање одштампаног маркера правом камером и стабилност сидрења — не може веродостојно проверити без правог телефона.

4.2 Потребна опрема на телефону

За покретање апликације неопходан је Android уређај сертификован за ARCore — Google одржава јавну листу подржаних модела чије су камере, сензори покрета и процесори прошли проверу квалитета праћења. Уређај мора имати Android 7.0 (API ниво 24) или новији, што се поклапа са вредношћу `minSdk = 24` у конфигурацији пројекта. Поред тога, на уређају мора бити инсталирана услуга *Google Play Services for AR* (ARCore), која се дистрибуира и аутоматски ажурира преко продавнице *Google Play*, па апликација не пакује ARCore у себе, већ се ослања на ту системску услугу.

Да би се апликација инсталирала директно из окружења *Android Studio*, на телефону је потребно укључити опције за програмере и дозволити отклањање грешака преко USB везе (*USB debugging*); алтернативно, новије верзије окружења подржавају и бежично упаривање преко Wi-Fi мреже. Пошто апликација податке преузима са сервера у локалној мрежи, телефон и рачунар на коме сервер ради морају бити повезани на исту Wi-Fi мрежу.

4.3 Креирање пројекта и зависности

Пројекат се креира кроз чаробњак у окружењу *Android Studio*, избором шаблона *Empty Views Activity*, са језиком *Kotlin* и конфигурацијом изградње у облику *Gradle Kotlin DSL* датотека (`build.gradle.kts`). Након креирања, у датотеку `app/build.gradle.kts` додају се зависности приказане у табели 2, чије су верзије централизоване у каталогу `gradle/libs.versions.toml`.

Зависност	Верзија
io.github.sceneview:arsceneview	2.2.1
androidx.core:core-ktx	1.16.0
androidx.appcompat:appcompat	1.7.0
com.google.android.material:material	1.12.0
androidx.constraintlayout:constraintlayout	2.2.1
androidx.lifecycle:lifecycle-runtime-ktx	2.8.7
com.squareup.retrofit2:retrofit	2.11.0
com.squareup.retrofit2:converter-kotlinx-serialization	2.11.0
org.jetbrains.kotlinx:kotlinx-serialization-json	1.8.0
com.squareup.okhttp3:logging-interceptor	4.12.0

Табела 2: Зависности коришћене у пројекту

Кључна зависност је библиотека *SceneView* (arsceneview), која обједињује ARCore и *Filament* механизам за рендеровање: уместо директног коришћења ARCore-a, што би захтевало ручно писање OpenGL кода за приказ камере и 3Д објеката, SceneView нуди готову компоненту ARSceneView, граф сцене са чворовима (Node) и рендеровање „из кутије”. За мрежну комуникацију са сервером користи се *Retrofit* у комбинацији са *kotlinx-serialization* конвертором, чиме се JSON одговори сервера аутоматски претварају у типизирание Kotlin објекте, док се *OkHttp logging-interceptor* укључује само у развојној варијанти ради увида у мрежни саобраћај. Корутине и *lifecycle-runtime-ktx* обезбеђују асинхронно периодично преузимање података усклађено са животним циклусом активности. У блоку android укључена је и опција buildFeatures { viewBinding = true }, којом се за сваки XML распоред генерише типизована класа за приступ елементима интерфејса, чиме се избегавају позиви findViewById.

4.4 Подешавање манифеста

Датотека AndroidManifest.xml мора прецизно описати захтеве апликације према систему. Пројекат декларише следеће ставке:

- Дозвола за приступ камери, која је обавезна, јер без камере проширена стварност није могућа, и дозвола за приступ интернету, неопходна за преузимање података са сервера:

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.INTERNET" />
```

- Декларација да је AP хардвер обавезан, у комбинацији са ознаком да је ARCore неизоставна компонента апликације:

```
<uses-feature
    android:name="android.hardware.camera.ar"
    android:required="true" />
<meta-data
    android:name="com.google.ar.core"
    android:value="required"
    tools:replace="android:value" />
```

Овим паром апликација постаје „AR Required”, што значи да се у продавници нуди само уређајима који подржавају ARCore. Ознака `tools:replace` над атрибутом `android:value` овде је неопходна: библиотека `SceneView` у свом манифесту исту мета-ознаку декларише са вредношћу `optional`, па би спајање манифеста (*manifest merge*) без ове ознаке завршило грешком због сукоба вредности; овако вредност из апликације надјачава вредност из библиотеке.

- Атрибут `android:usesCleartextTraffic="true"` на елементу `<application>`, којим се дозвољава нешифрован HTTP саобраћај. Од Android-a 9 систем подразумевано блокира саобраћај без TLS-a, а сервер из овог рада ради у локалној мрежи на адреси облика `http://192.168.0.49:8080`, без сертификата, па је изузетак неопходан за демонстрацију.

4.5 Кораци до прве AR сцене

Када су окружење, зависности и манифест спремни, до прве функционалне AR сцене долази се кроз неколико корака:

1. У XML распоред активности додаје се компонента `ARSceneView`, која истовремено приказује слику са камере и рендероване 3Д објекте.
2. Поглед на сцену повезује се са животним циклусом активности доделом `sceneView.lifecycle = lifecycle`, чиме се AR сесија аутоматски паузира и наставља заједно са активношћу.
3. У позиву `configureSession` конфигурише се сесија: гради се база слика `AugmentedImageDatabase` са маркерима учионица и додељује пољу `config.augmentedImageDatabase`, чиме се укључује праћење слика описано у одељку 2.5.
4. Подразумевани приказ детектованих равни искључује се поставком својства `isEnabled` компоненте `planeRenderer` на вредност нетачно, јер апликацији равни нису потребне, а истакнуте површине би визуелно оптерећивале екран.
5. Преко повратног позива `onSessionUpdated` апликација се укључује у обраду сваког фрејма и из њега чита новопрепознате и ажуриране слике маркера.
6. У време извршавања тражи се дозвола за камеру, будући да се од API нивоа 23 опасне дозволе морају експлицитно одобрити од стране корисника.
7. Апликација се покреће на физичком уређају, са одштампаним маркером, где се проверава да ли се маркер препознаје и да ли панел стабилно стоји изнад њега.

У наредном поглављу приказано је како се од ових основних градивних елемената — сесије, базе слика, сидара и чворова — гради комплетна функционалност AR прегледа учионица.

5 Имплементација АР прегледа учионица

Централна компонента апликације је активност `AugmentedRealityActivity`, у којој је смештена целокупна АР логика. Активност користи компоненту `ARSceneView` из библиотеке `SceneView`, која обједињује АР сесију библиотеке `ARCore` и графички приказ виртуелних објеката преко слике са камере. Пошто у једној сесији може истовремено да буде праћено више учионица, активност све ресурсе води у мапама кључаним именом препознате слике: мапу сидришних чворова, мапу панела и мапу послова за периодично преузимање података. У наставку је описано како апликација користи кључне функције `ARCore` и `SceneView` интерфејса: од конфигурације сесије и базе маркера, преко препознавања учионице и сидрења панела, до исцртавања података и ослобађања ресурса.

5.1 Конфигурација сесије и базе слика

База референтних слика гради се у позиву `configureSession`, при сваком покретању АР сесије. Апликација из фасцикле `assets/markers` чита све PNG датотеке и сваку додаје у базу под именом које одговара називу датотеке без екстензије:

```
configureSession { session, config ->
    val augmentedImageDatabase = AugmentedImageDatabase(session)
    val markerFileNames = assets.list("markers").orEmpty()
        .filter { it.endsWith(".png") }
    markerFileNames.forEach { markerFileName ->
        val imageName = markerFileName.substringBeforeLast('.')
        try {
            assets.open("markers/$markerFileName").use { markerStream ->
                val markerBitmap = BitmapFactory.decodeStream(markerStream)
                if (markerBitmap != null) {
                    augmentedImageDatabase.addImage(
                        imageName,
                        markerBitmap,
                        MARKER_PHYSICAL_WIDTH_METERS
                    )
                }
            }
        } catch (exception: Exception) {
            Log.w("AugmentedRealityActivity",
                "Preskočen marker $markerFileName", exception)
        }
    }
    config.augmentedImageDatabase = augmentedImageDatabase
}
```

Трећи параметар функције `addImage` је физичка ширина одштампаног маркера, константа `MARKER_PHYSICAL_WIDTH_METERS` вредности `0.21f` — маркер се штампа на папиру формата А4, чија је ширина 21 центиметар. Захваљујући томе `ARCore` одмах по препознавању тачно процењује позу и удаљеност маркера. Сваки позив `addImage` обавијен је у `try/catch` блок: ако нека слика нема довољно карактеристичних тачака, `ARCore` одбија да је упише у базу изузетком, па се таква слика прескаче уз упис у дневник

уместо да обори читаву сесију. Ако ниједна слика не уђе у базу, кориснику се исписује порука „Nema markera u bazi aplikacije.”. Поред конфигурације сесије, искључен је и подразумевани приказ детектованих равни поставком `planeRenderer.isEnabled = false`, јер апликација равни не користи.

5.2 Препознавање учионице

Преко повратног позива `onSessionUpdated` апликација се укључује у обраду сваког фрејма AR сесије. Из фрејма се функцијом `getUpdatedAugmentedImages` читају слике чије се стање у том фрејму променило:

```
onSessionUpdated = { _, frame ->
    frame.getUpdatedAugmentedImages().forEach { augmentedImage ->
        when {
            augmentedImage.isTracking ->
                onAugmentedImageTracking(augmentedImage)
            augmentedImage.trackingState == TrackingState.STOPPED ->
                onAugmentedImageStopped(augmentedImage.name)
        }
    }
}
```

Слика у стању `TRACKING` активно се прати и њена поза је поуздана, па се за њу поставља панел; стање `STOPPED` значи да је праћење трајно прекинуто, па се сви ресурси везани за ту слику ослобађају. Идентификатор учионице изводи се из имена препознате слике: маркер `ucionica_101` даје идентификатор `101` изразом `imageName.substringAfterLast('_')`. Тај идентификатор се затим користи у путањи серверског позива, а кориснику се у статусној траци исписује порука „Učionica 101 — podaci uživo.”.

5.3 Сидрење панела

Када слика први пут уђе у стање праћења, апликација за њу креира сидро у центру маркера и обавија га у чвор сцене:

```
val anchor = augmentedImage.createAnchor(augmentedImage.centerPose)
val anchorNode = AnchorNode(binding.sceneView.engine, anchor)
binding.sceneView.addChildNode(anchorNode)
panelOffsets[imageName] =
    -(augmentedImage.extentZ / 2f + PANEL_BOTTOM_GAP_METERS +
      PANEL_HEIGHT_METERS / 2f)
```

Сидро везано за позу слике обезбеђује да садржај остане причвршћен за маркер док `ARCore` прецизира своје разумевање сцене. Сам панел је чвор типа `ImageNode` — раван правоугаоник на који се као текстура пресликава битмапа — величине 32 пута 24 центиметра у стварном простору:

```
val panelNode = ImageNode(
```

```

        materialLoader = binding.sceneView.materialLoader,
        bitmap = panelBitmap,
        size = Size(PANEL_WIDTH_METERS, PANEL_HEIGHT_METERS, 0f)
    )
    panelNode.rotation = Rotation(x = -90f, y = 0f, z = 0f)
    panelNode.position = Position(
        0f, 0f, panelOffsets[imageName] ?: DEFAULT_PANEL_OFFSET_METERS
    )
    anchorNode.addChildNode(panelNode)

```

Координатни систем препознате слике поставља X осу дуж њене ширине, Z осу дуж висине, а Y осу управно на њу, па се ротацијом од -90 степени око X осе правоугаоник панела поравнава са равни маркера. Померај дуж негативне Z осе, израчунат приликом препознавања, износи половину висине маркера ($\text{extentZ} / 2$) увећану за размак од три центиметра и половину висине самог панела — тиме панел лебди тачно изнад горње ивице маркера, односно изнад врата учионице.

5.4 Исцртавање панела

Садржај панела не моделује се као 3Д геометрија, већ се користи постојећи систем Android распореда: класа `PanelBitmapRenderer` надувава XML распоред `view_room_panel.xml`, попуњава га подацима и исцртава у битмапу. Панел приказује назив учионице, ред заузетости („Zauzeta do 10:00 — Pametna okruženja” или „Slobodna”), три реда са вредностима температуре, буке и угљен-диоксида уз кружне индикаторе обојене према статусу (зелено за OK, жуто за WARNING, црвено за CRITICAL) и ред са препоруком сервера, који се при критичном статусу такође боји црвено. Попуњени распоред се затим мери и исцртава у фиксној резолуцији:

```

panelView.measure(
    View.MeasureSpec.makeMeasureSpec(
        BITMAP_WIDTH, View.MeasureSpec.EXACTLY),
    View.MeasureSpec.makeMeasureSpec(
        BITMAP_HEIGHT, View.MeasureSpec.EXACTLY)
)
panelView.layout(0, 0, BITMAP_WIDTH, BITMAP_HEIGHT)
val bitmap = Bitmap.createBitmap(
    BITMAP_WIDTH, BITMAP_HEIGHT, Bitmap.Config.ARGB_8888
)
panelView.draw(Canvas(bitmap))

```

Константе `BITMAP_WIDTH` и `BITMAP_HEIGHT` износе 1024 и 768 пиксела, што при односу страница 4:3 одговара физичким димензијама панела од 32 пута 24 центиметра. Када за већ приказану учионицу стигну нови подаци, чвор се не ствара изнова: довољно је постојећем `ImageNode` објекту доделити нову вредност својства `bitmap`, чиме `SceneView` замењује текстуру на месту, без трзаја у сцени.

5.5 Преузимање података у реалном времену

Комуникација са сервером изведена је библиотеком *Retrofit* уз *kotlinx-serialization* конвертор. Мрежни интерфејс садржи две суспендоване функције:

```
interface RoomApiService {

    @GET("api/rooms")
    suspend fun getRooms(): List<RoomResponse>

    @GET("api/rooms/{roomId}")
    suspend fun getRoom(@Path("roomId") roomId: String): RoomResponse
}
```

Одговори сервера пресликавају се у типизирани објект: класа `RoomResponse` означена аномацијом `@Serializable` садржи тачно она поља која сервер враћа (назив, заузетост, распоред, вредности сензора, статусе и препоруку), при чему су статуси моделовани набрајањем `SensorStatus` са вредностима `OK`, `WARNING` и `CRITICAL`. Адреса сервера чита се из дељених подешавања (`SharedPreferences`), где је уписује почетни екран.

За сваку праћену учионицу покреће се по један посао (`Job`) у опсегу `lifecycleScope`, који у петљи преузима стање и освежава панел на сваке три секунде:

```
pollingJobs[imageNames] = lifecycleScope.launch {
    while (isActive) {
        try {
            val room = roomApiService.getRoom(roomId)
            val panelBitmap = panelBitmapRenderer.render(
                this@AugmentedRealityActivity, room
            )
            updatePanel(imageNames, panelBitmap)
        } catch (exception: IOException) {
            setStatusText("Server nije dostupan. Proveri adresu servera.")
        } catch (exception: HttpException) {
            setStatusText("Server nije dostupan. Proveri adresu servera.")
        }
        delay(POLLING_INTERVAL_MILLISECONDS)
    }
}
```

Мрежне грешке не прекидају петљу: кориснику се исписује порука о недоступном серверу, а следећа итерација поново покушава преузимање, па се панел сам опоравља чим сервер постане доступан. Посао се отказује чим праћење слике престане, као и у методи `onPause`, чиме се спречава мрежни саобраћај док је апликација у позадини; у методи `onResume` послови се поново покрећу за све учионице које су и даље усидрене.

5.6 Животни циклус и ослобађање ресурса

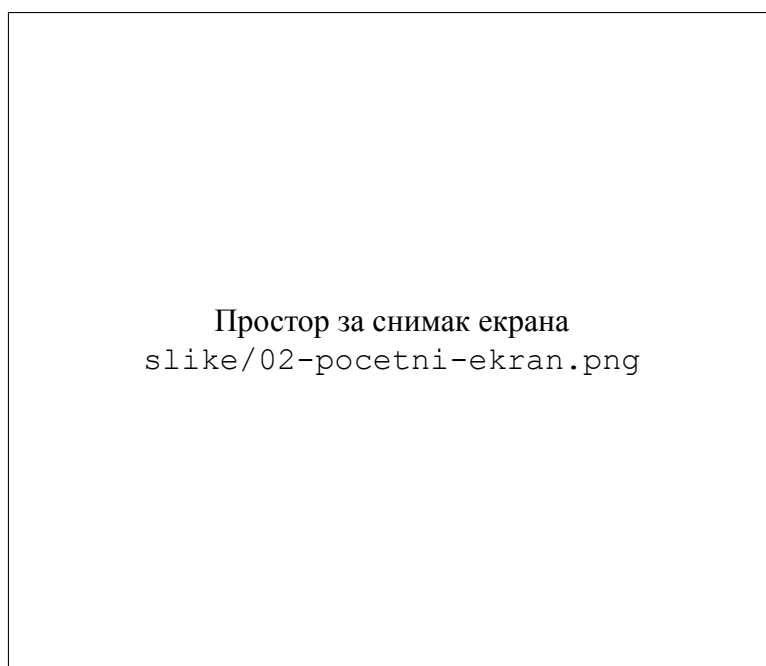
Када `ARCore` праћење неке слике пређе у стање `STOPPED`, није довољно само сакрити панел: чвор панела се уклања из графа сцене и уништава позивом `destroy()`, а затим се исто чини и са сидришним чвором, чиме се ослобађа и само `ARCore` сидро. Овај корак је важан, јер свако активно сидро троши ресурсе праћења у сваком фрејму — `ARCore` његову позу непрестано ажурира — па би нагомилавање неослобођених сидара временом деградирало перформансе сесије. Истим редоследом ослобађају се и припадајуће ставке у мапама и отказује посао за преузимање података, чиме се гарантује да по престанку праћења не остаје ниједан активан ресурс везан за ту учионицу.

6 Приказ апликације

У овом поглављу апликација је представљена кроз приказ њених екрана, редоследом којим се корисник са њима сусреће при типичној употреби. За сваки екран дат је кратак опис садржаја и могућих интеракција, праћен снимком екрана. Кориснички интерфејс изведен је у складу са Материјал дизајн (енгл. *Material Design*) смерницама, уз употребу компоненти библиотеке `Material Components` за Андроид.

6.1 Почетни екран

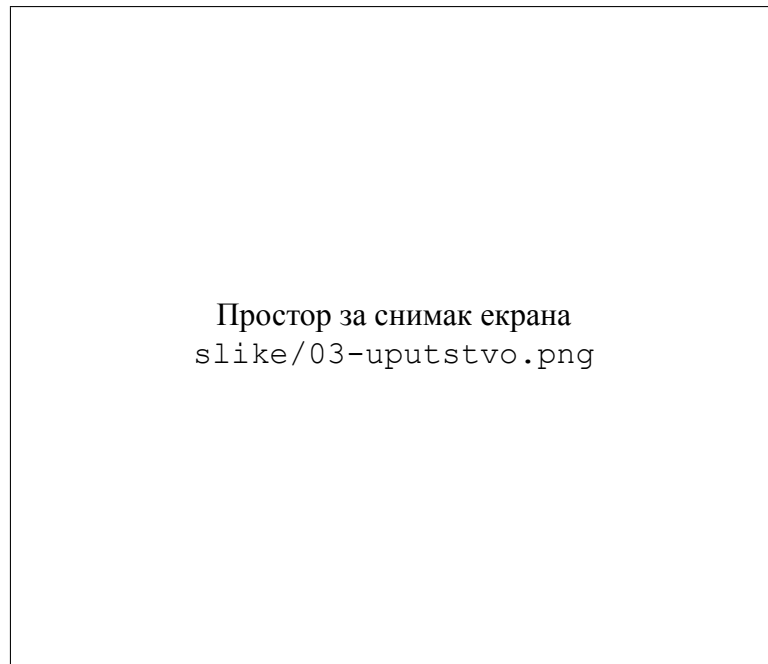
Почетни екран представља улазну тачку апликације. У горњем делу приказани су логотип у заобљеном блоку, назив `AR Pametne Učionice` и кратак опис који кориснику објашњава да се усмеравањем камере на маркер учионице изнад њега приказују живи подаци о просторији. Испод описа налази се текстуално поље за адресу сервера, унапред попуњено вредношћу `http://192.168.0.49:8080`; свака измена се одмах трајно чува у дељеним подешавањима, па се при следећем покретању не мора уносити поново. На дну екрана су два дугмета: главно дугме `Pokreni AR pregled` отвара АР екран, док дугме `Uputstvo` води на екран са упутством.



Слика 2: Почетни екран апликације

6.2 Екран са упутством

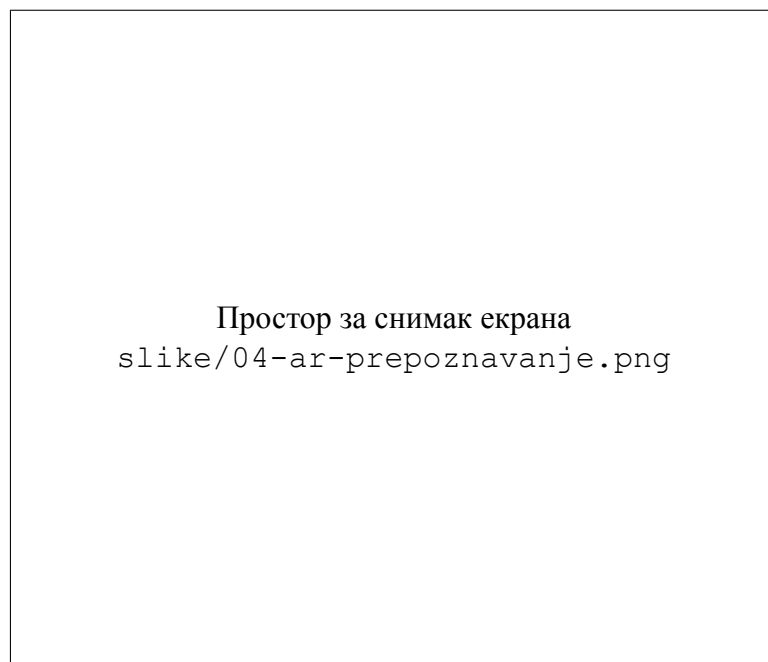
Екран са упутством садржи четири нумерисана корака приказана у виду засебних картица. Први корак упућује корисника да пронађе маркер на вратима учионице, други да камеру телефона усмери ка маркеру, трећи објашњава да се изнад маркера приказује панел са живим подацима о просторији, а четврти да боје индикатора показују статус измерених вредности — зелена да је све у реду, жута упозорење, а црвена критично стање.



Слика 3: Екран са упутством за коришћење

6.3 АР екран — препознавање маркера

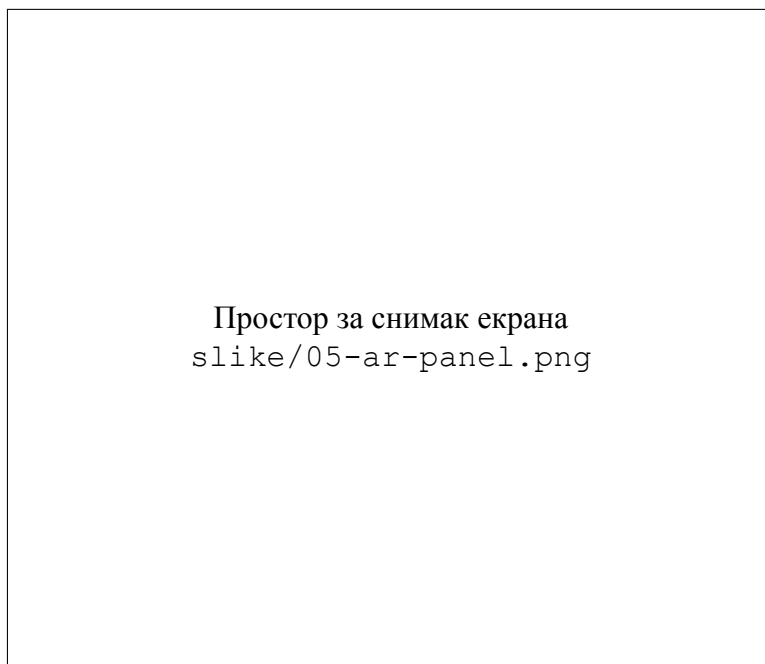
Након покретања АР прегледа отвара се екран на коме слика са камере испуњава целу површину приказа. На врху се налази тамна трака са дугметом за затварање и насловом апликације, а испод ње статусна порука у облику пилуле која у почетном стању гласи `Pronađi marker učionice.` и која корисника током рада води кроз процес. У овој фази корисник полако помера телефон док камера не обухвати одштампани маркер на вратима учионице.



Слика 4: АР екран пре препознавања маркера

6.4 AP екран — усидрени панел

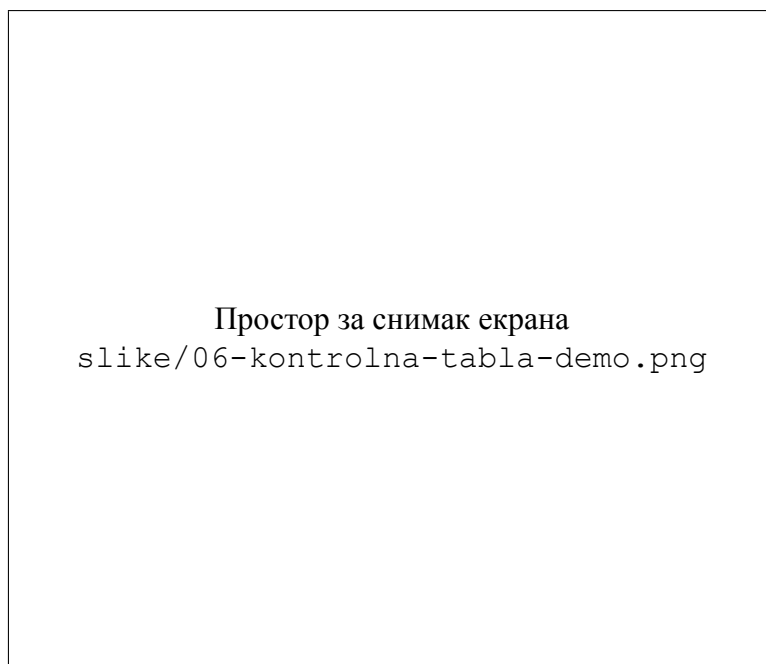
Чим ARCore препозна маркер, изнад његове горње ивице појављује се усидрени панел са подацима о учионици, а статусна порука се мења у `Učionica 101 — podaci uživo..` Панел приказује назив просторије, ред заузетости са називом текућег часа и временом до којег је учионица заузета, три реда са температуром, буком и угљен-диоксидом уз индикаторе обојене према статусу и препоруку сервера. Панел остаје причвршћен за маркер док се корисник креће: може му се прићи, одмаћи се или га посматрати са стране, а садржај се освежава на сваке три секунде свежим подацима са сервера.



Слика 5: Усидрени панел изнад маркера са живим подацима

6.5 Демонстрација са контролном таблом

Завршни део демонстрације повезује све три компоненте система. Презентер на контролној табли помери клизач — на пример, подигне вредност угљен-диоксида изнад критичног прага — и промена се после задршка од 300 милисекунди шаље серверу. Већ при следећем преузимању, најкасније три секунде касније, AP панел на телефону приказује нову вредност, индикатор квалитета ваздуха прелази у црвено и исписује се препорука `Hitno provetriti prostoriju..`, без иједне акције на самом телефону.



Слика 6: Промена на контролној табли одражена на АР панелу

Приказани екрани заједно чине заокружену целину: од почетног екрана и упутства, преко препознавања маркера, до живог панела који у реалном времену одражава стање паметног окружења. Демонстрациони ток — клизач на контролној табли, упис на серверу, освежен панел у проширеној стварности у року од три секунде — показује да је архитектура спремна и за стварну примену: довољно је да уместо контролне табле исте крајње тачке позивају прави сензори уграђени у учионице, без иједне измене у АР апликацији. Као могућа проширења у будућем раду издвајају се повезивање стварних сензора температуре, буке и угљен-диоксида преко постојећег интерфејса, додавање маркера за већи број учионица и зграда, као и интеракција на самом панелу, попут резервације слободне учионице додиром.